```
  return webClient
    .get()
    .uri(BOOKS_ROOT_API + isbn)
    .retrieve()
    .bodyToMono(Book.class)
    .timeout(Duration.ofSeconds(3), Mono.empty())    ❶
}
...
```

❶ The fallback returns an empty Mono object.

NOTE In a real production scenario, you might want to externalize the timeout configuration by adding a new field to the `ClientProperties`. In that way, you can change its value depending on the environment without having to rebuild the application. It's also essential to monitor any timeout and tune its value if necessary.

UNDERSTANDING HOW TO USE TIMEOUTS EFFECTIVELY

Timeouts improve application resilience and follow the principle of failing fast. But setting a good value for the timeout can be tricky. You should consider your system architecture as a whole. In the previous example, you defined a 3-second timeout. This means that a response should get from Catalog Service to Order Service within that time limit. Otherwise, either a failure or a fallback occurs. Catalog Service, in turn, sends a request to the PostgreSQL database to fetch the data about the specific book and waits for a response. A connection timeout guards that interaction. You should carefully design a time-limiting strategy for all the integration points in your system to meet your software's SLAs and guarantee a good user experience.

If Catalog Service were available, but a response couldn't get to Order Service within the time limit, the request would likely still be processed by Catalog Service. That is a critical point to consider when configuring timeouts. It doesn't matter much for read or query operations because they are idempotent. For write or command operations, you want to ensure proper